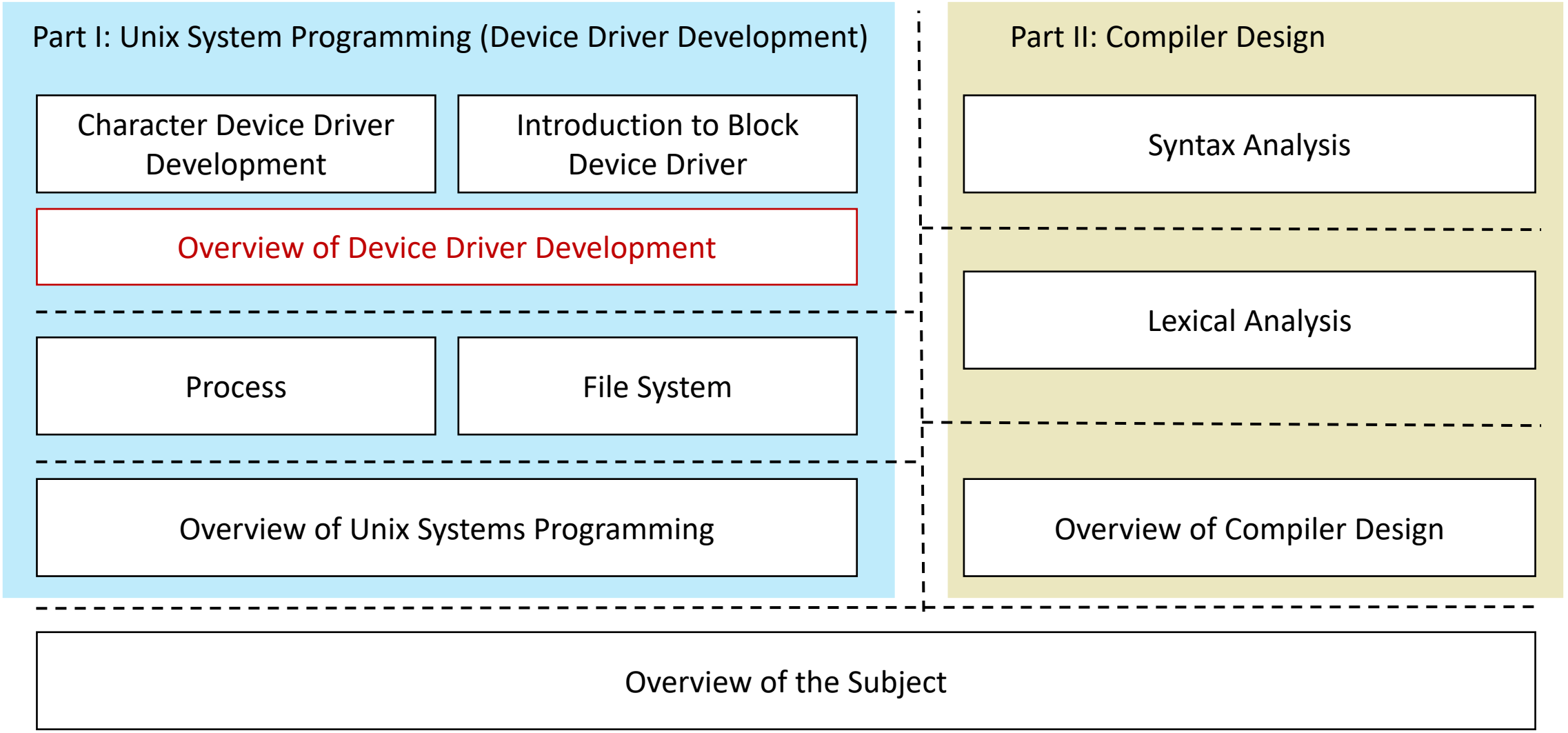
COMP3438 System Programming

Introduction to Device Drivers

Course Organization



Outline

What is a device driver?

Related OS infrastructure

Devices and files

Major design issues

Types of device drivers

What is a device driver

Device driver is a special kind of library, which can be loaded into the OS kernel, and links application program with the I/O devices

Related OS infrastructure

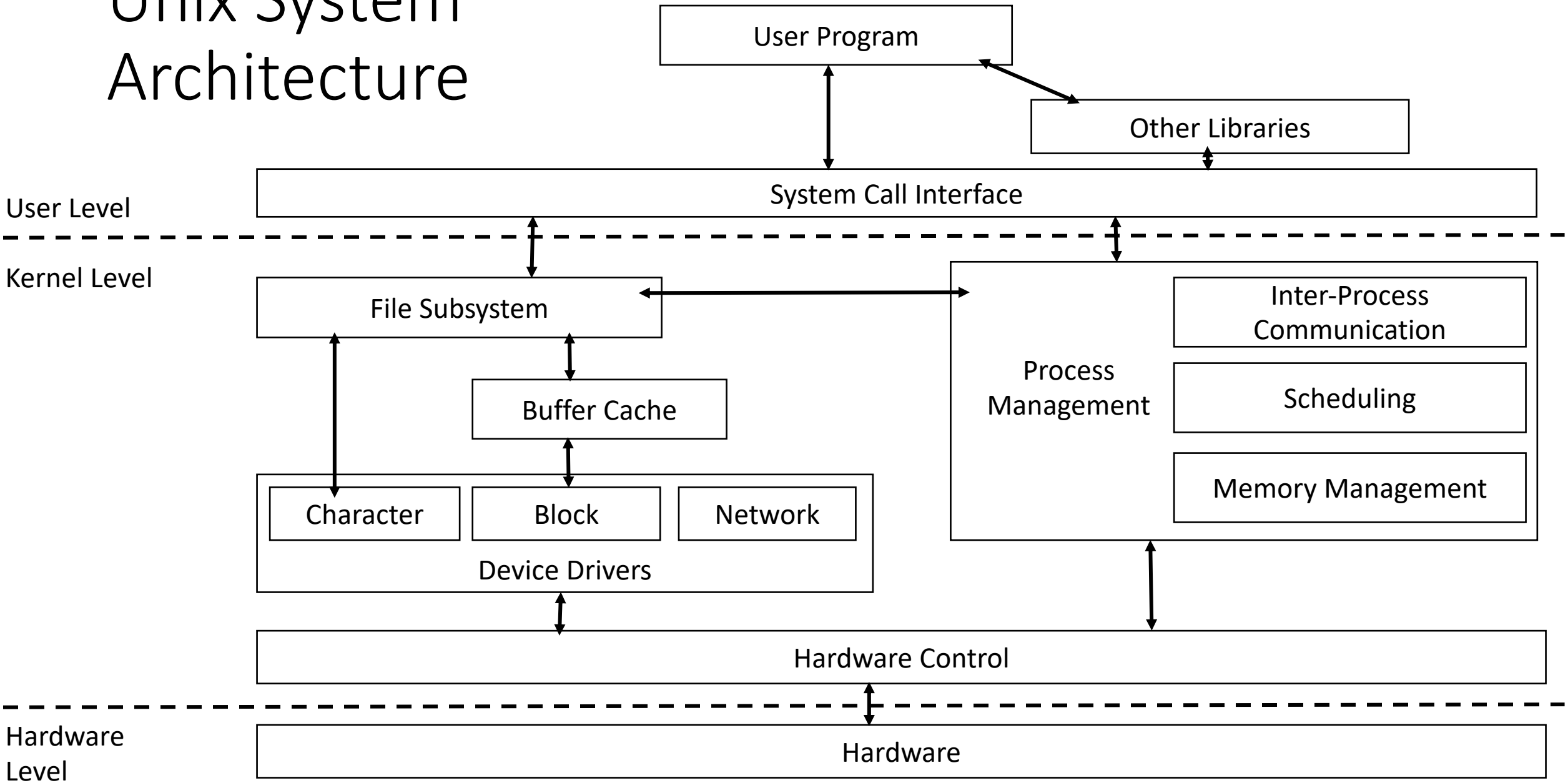
In Unix, the following infrastructure is related to device drivers:

Unix system architecture

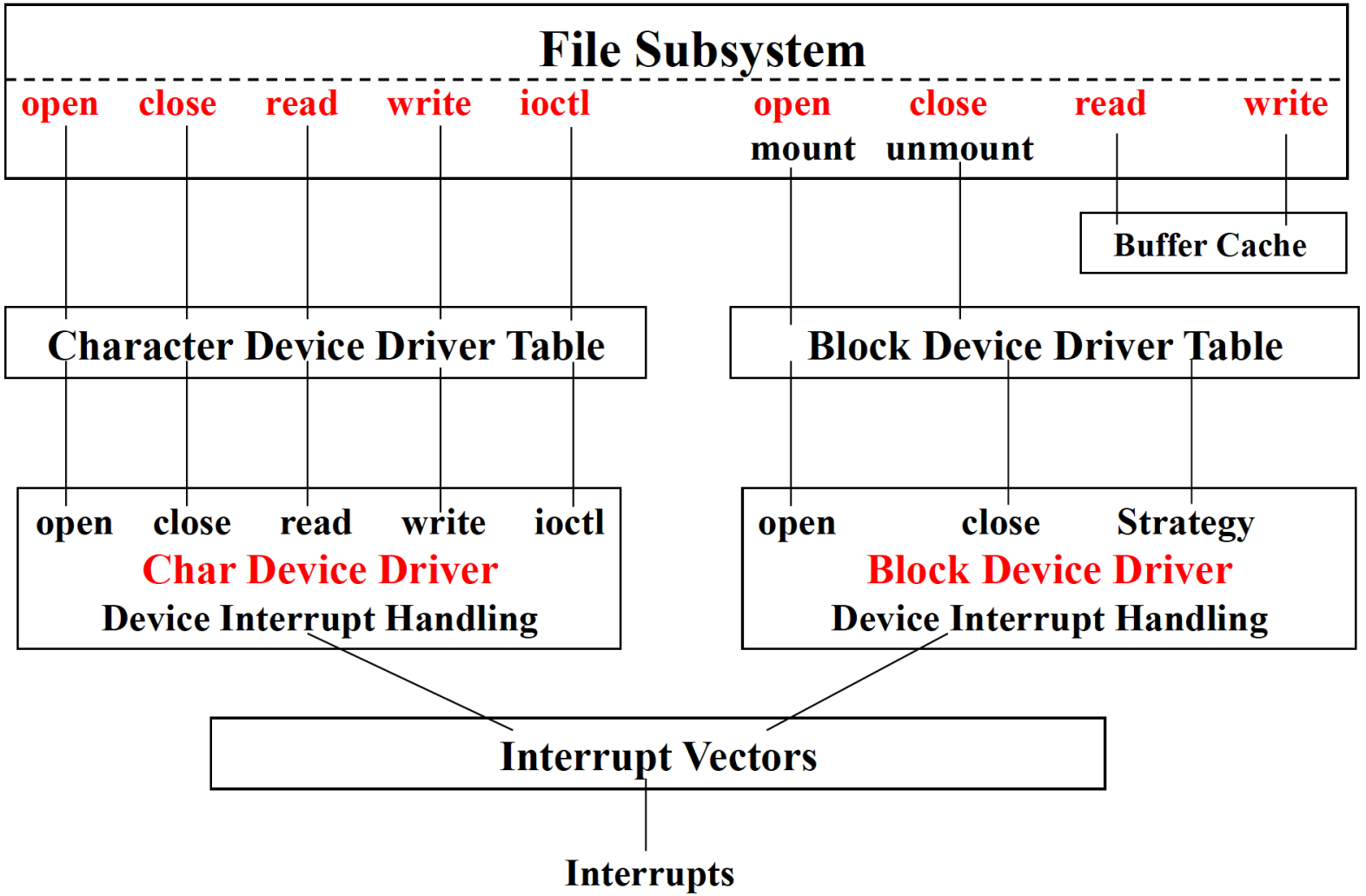
File subsystem and its relations with char/block device driver tables

Char/block device driver tables

Unix System Architecture



File subsystem & char/block device driver tables



Char/block device driver tables

Character Device Driver Table

	open	close	read	write	ioctl
0	conopen	conclose	conread	conwrite	conioctl
1	testopen	testclose	testread	testwrite	nodev
3	dzbopen	dzbclose	dzbread	dzbwrite	dzbioctl
4	syopen	nulldev	syread	sywrite	syioctl
5	nulldev	nulldev	mmread	mmwrite	nodev

Block Device Driver Table

open	close	Strategy
gdopen	gdclose	gdstrategy
rdopen	rdclose	rdstrategy

Major Number: **The index (id) of a device driver in char/block device driver tables.**
Minor Number: **A number is used inside a device driver, e.g., one driver may control many devices then minor number can be used to distinguish them.**

Advantages to separate device drivers from the OS

OS designers

- Devices may not be available when an OS is designed.
- Do not need to worry about how to operate devices (set up registers, check statuses, …).
- Focus on OS itself by providing a generic interface for device driver development.

Device driver designers

- Do not need to worry about how I/O is managed in OS (how to design kernel data structures to efficiently operate devices, …)
- Focus on implementing functions of devices with device-related commands following the generic I/O interface

Devices and Files

In Unix, devices are treated as special files.

A file is associated with an inode. We can use

```
mknod <file_name> <c or b> <major_number> <minor_number>
```

to create a special file for a device file.

When we create a special file (an inode) for a device file, we associate <major, minor> number with the file (the inode).

The major number associated with a special file is used to identify its corresponding device driver instance in the kernel.

Character/Block Device Files

Basically, there are two types of device files: character device file and block device file.

Example:

```
ls -l /dev/null
crw-rw-rw-  1  root  root  1  3  June 1, 1970  null
```

character device file

major number

minor number

```
ls -l /dev/bon
br--r--r--  1  root  root  97  0  June 1, 1970  0
br--r--r--  1  root  root  97  1  June 1, 1970  1
```

block device file

major number

minor number

Major Design Issues

OS / Device Driver Communication

Device Driver / Hardware Communication

Device Driver Operations

- Interpret commands received from OS
- Schedule multiple requests for services
- Manage data transfer across both interfaces
- Accept & process hardware interrupts
- Maintain the integrity of the device's and kernel's data structures

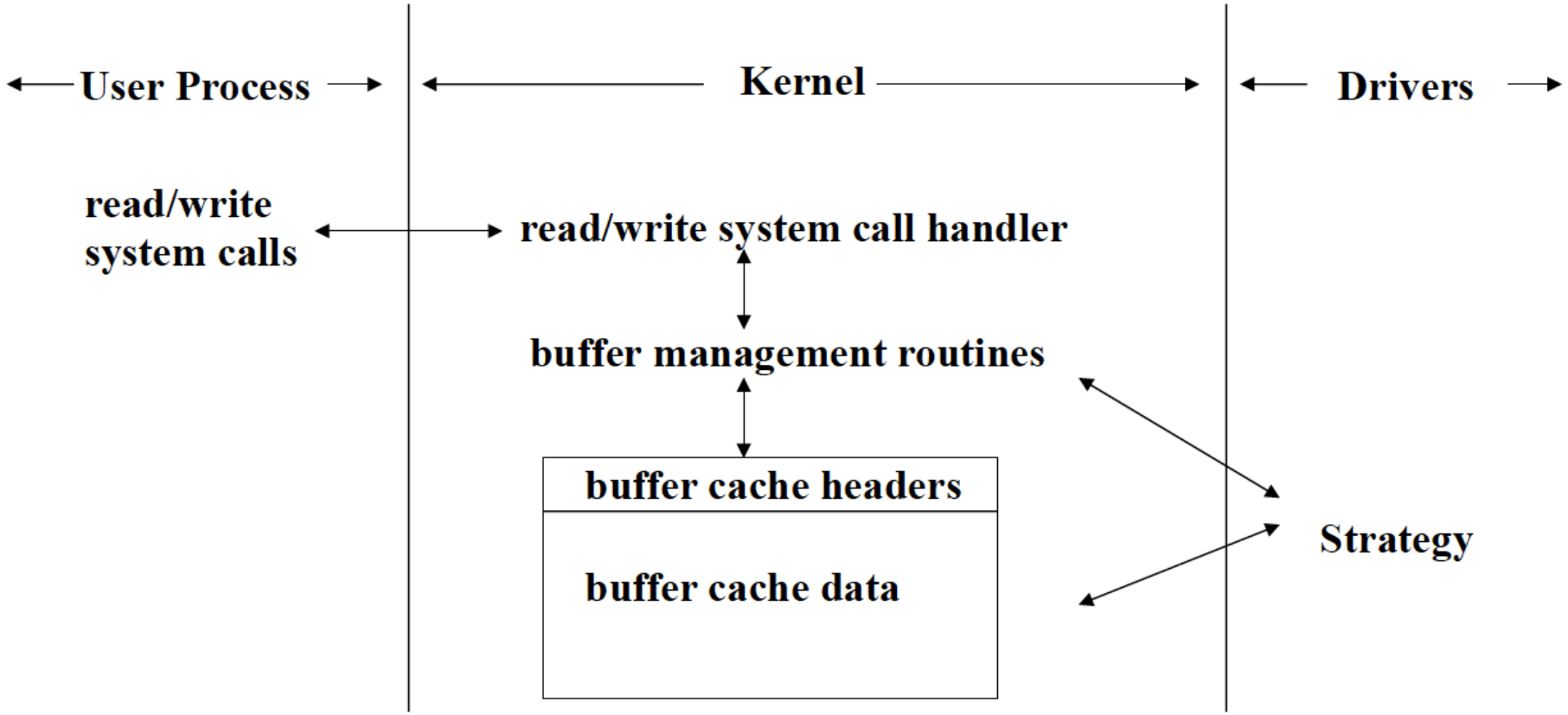
Types of Device Drivers

Based on the differences in the way that device drivers communicate with Linux

- Block
- Character
- Network

Block Device Drivers

Communicate with the OS through a collection of fixed size buffers



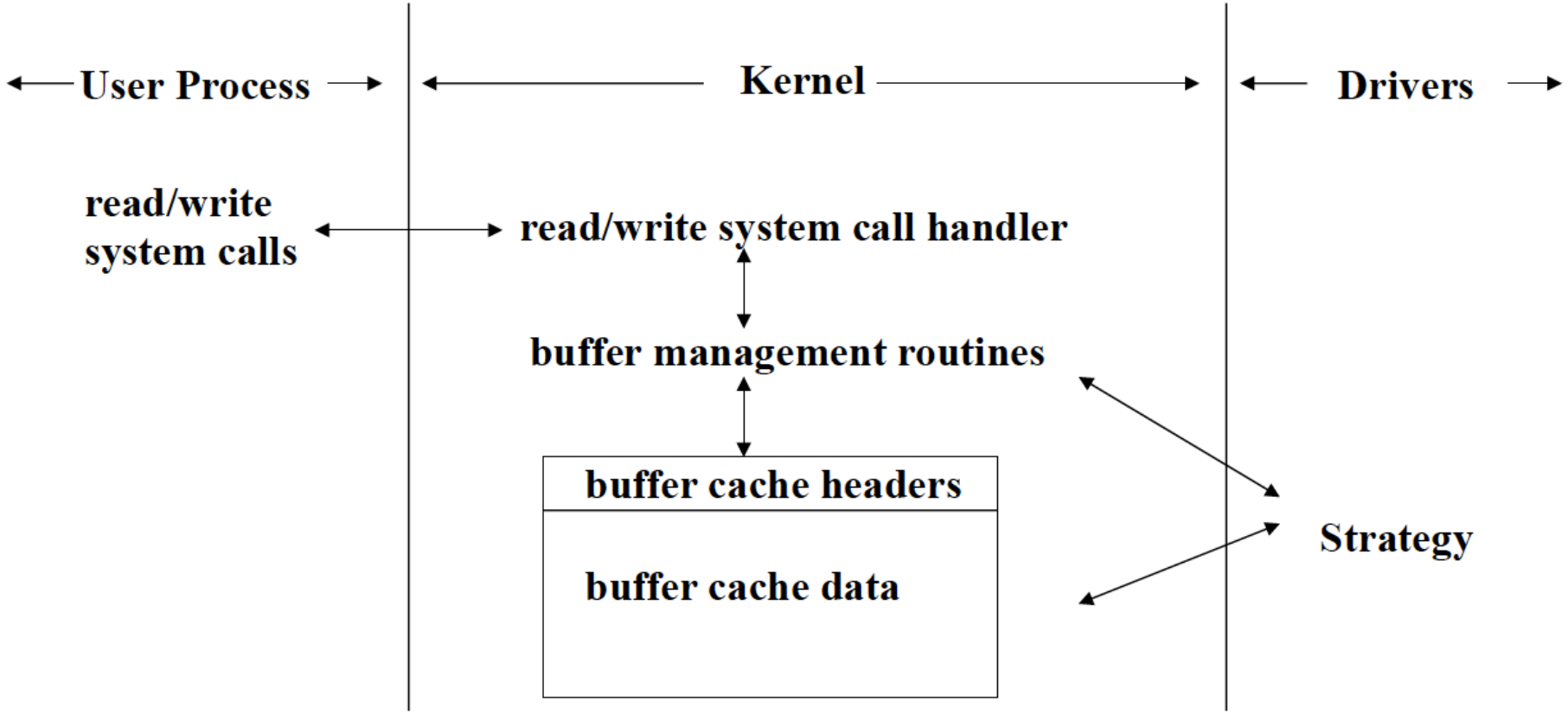
Block Device Drivers

OS manages a buffer cache; satisfy user requests for data by accessing buffers in the cache.

Driver is invoked when requested data is not in the cache; when buffers in the cache have been changed and must be written out (write back to the devices).

By using buffer cache, block drivers are insulated from the many details of user requests; only need to handle requests from the OS to fill or empty fixed size buffers.

Mainly support devices that contain file systems.

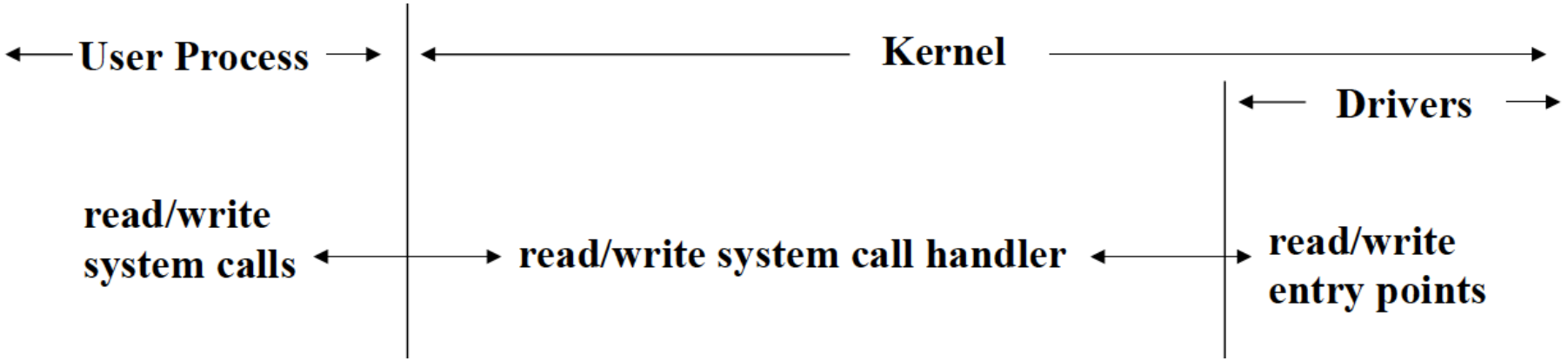


Character Device Drivers

Character devices can handle I/O requests of arbitrary size; support almost any type of devices.

Be used to handle data a byte at a time (e.g. keyboard); or work best with data in chunks smaller or larger than the standard fixed size buffer used by device driver (e.g. ADC)

The communication structure of character device driver:



Major differences between block/character drivers

Block driver – only interacts with buffer cache

Character driver – directly interacts with user requests from user processes

- I/O requests are directly passed (essentially unchanged) to the drivers from the user processes
- Character driver is responsible for transferring data directly to/from the kernel memory space and the user memory space.

General Programming Considerations

Device drivers are parts of the kernel and not normal user processes, which means we can only use the kernel routines in device driver programs, particularly C library functions or system calls for user level cannot be used.

Some kernel routines may have the same names as C library functions, but they are of totally different implementations.

Make frugal use of stack (local arrays & recursive function calls), as the stack space in the kernel is limited and not expandable.

Be aware of floating-point arithmetic: may cause incorrect results

Be aware of busy wait: may lock up the whole system

Summary

A device driver is a computer program that glues OS and its I/O devices together.

A device is treated as a special file in Unix.

There are three types of device drivers in Linux based on the different communication manner between device drivers and OS: block, character, and network.